



Efficient Inference with Transformers: Intro to Quantization

April 27, 2026

Recap: RAM requirements for LLMs

- I have a 7 billion parameter model. How much RAM do I need to host it?

- RAM Usage = 7 billion params × 2 bytes per parameter = 14 billion bytes*

– Kilo bytes = $2^{10} = 1,024$ ≈ 1 thousand bytes

– Mega bytes = $2^{20} = 1,048,576$

– Giga bytes = $2^{30} = 1,073,741,824$ ≈ 1 billion bytes

– Tera bytes = $2^{40} = 1,099,511,627,776$ ≈ 1 trillion bytes

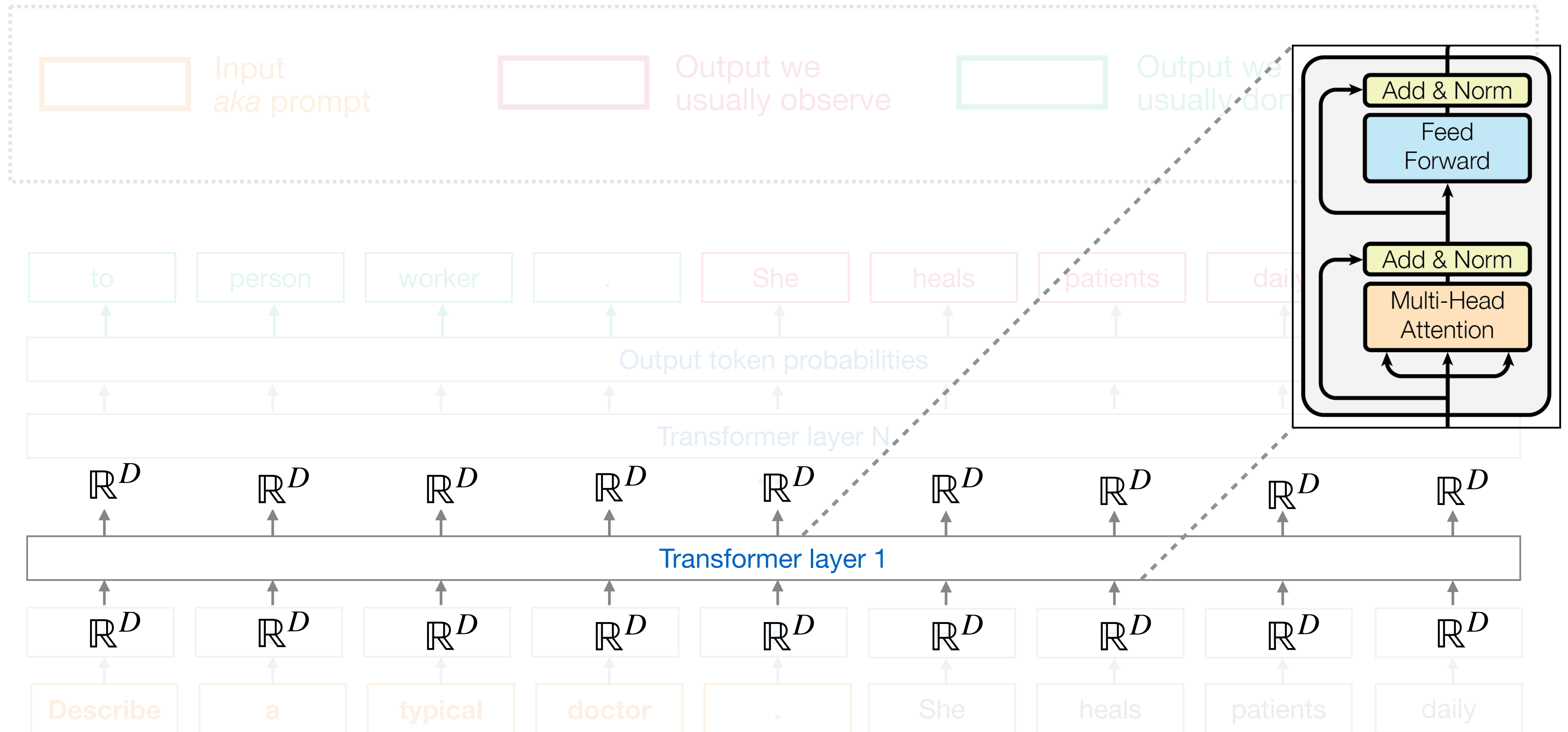
**One of the biggest limitations in running LLMs.
How can we reduce the memory footprint?**

This lecture: Representing parameters more efficiently

- Unlike regular ML models where a parameter is 4 bytes (32 bit float), LLMs are usually 2 bytes

* Actually we need a bit more for the intermediate computations

Recap: Every layer consists of large parameter matrices

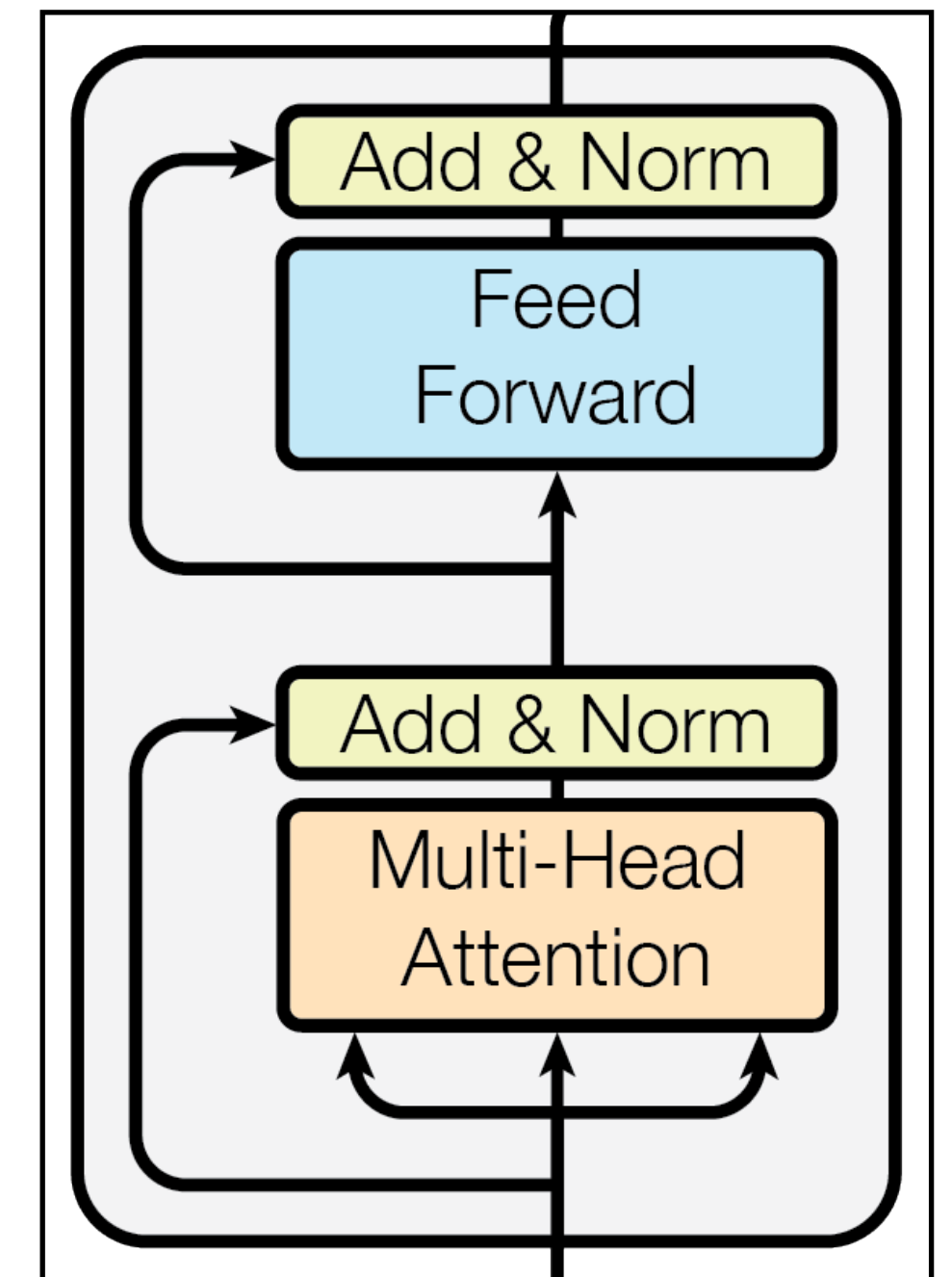


A closer look at model parameters

- Most computations within the model consist of

$$\text{output} = \text{input} \times \mathbf{W} + \mathbf{b}$$

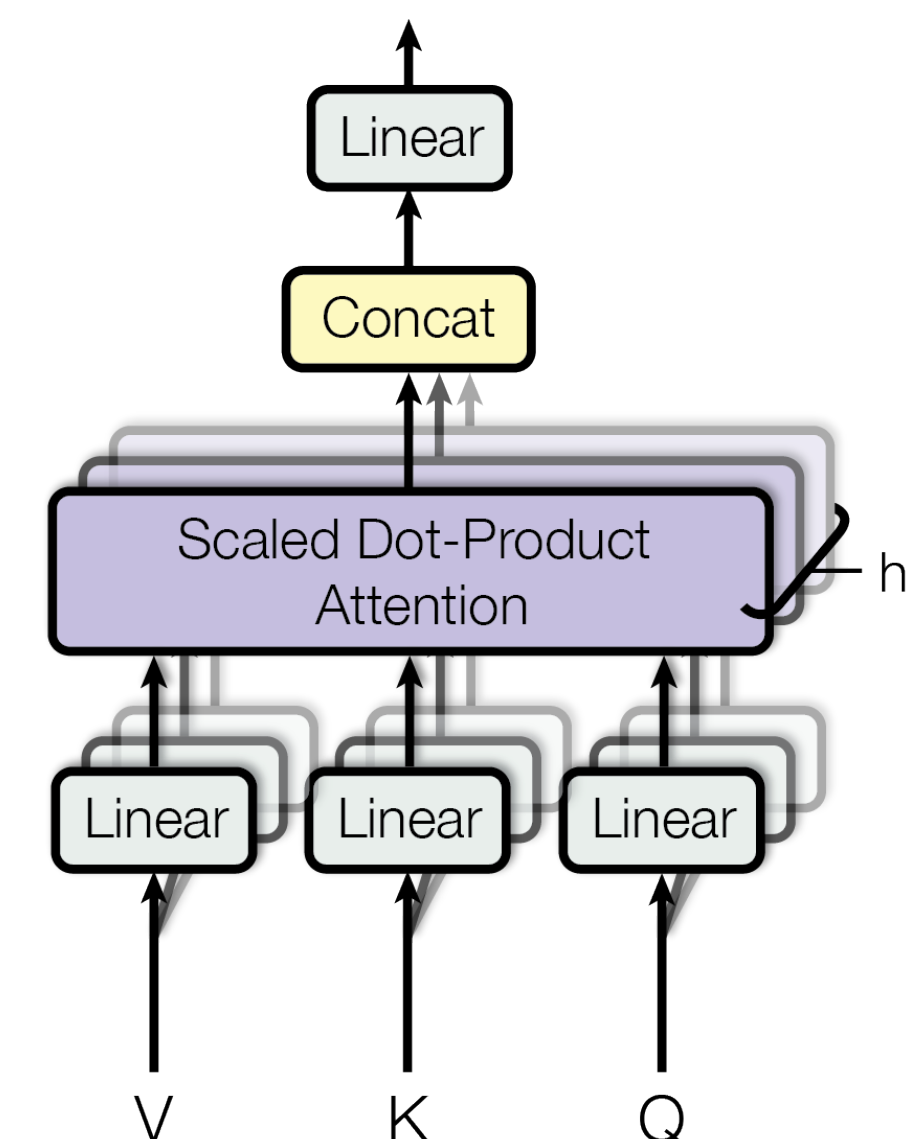
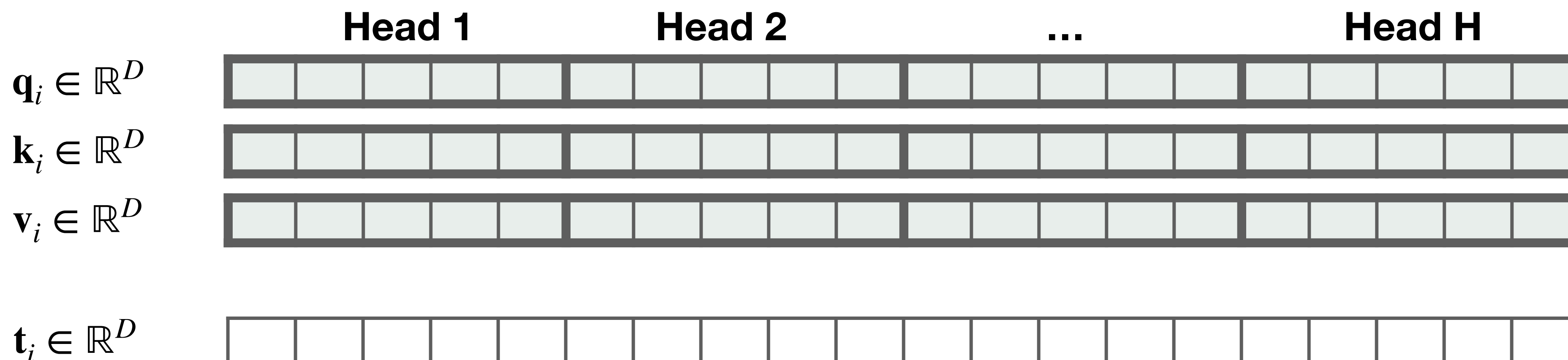
- $\mathbf{W} \in \mathbb{R}^{D_1 \times D_2}$ and $\mathbf{b} \in \mathbb{R}^{D_2}$ are the model parameters
- Sidenote on notation
 - Matrices are bold and capital letters ($\mathbf{W}$)
 - Vectors are bold and small letters ($\mathbf{b}$)
 - Scalars are regular (not bold) and capital letters (D)



Example # 1: Q, K and V for attention

$$\text{output} = \text{input} \times \mathbf{W} + \mathbf{b}$$

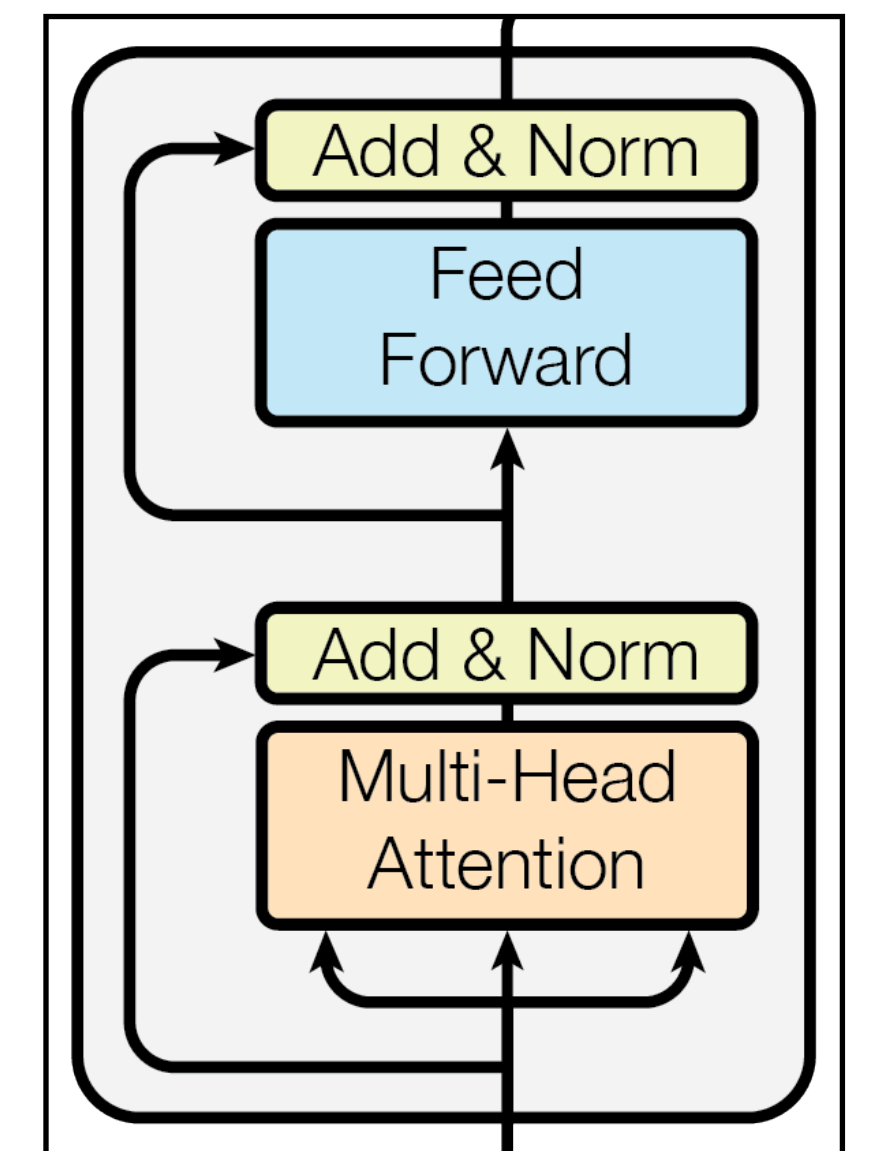
- Computing query ($\mathbf{q}$), key ($\mathbf{k}$) and value ($\mathbf{v}$) in i^{th} layer
- $\mathbf{q}_i = \mathbf{t}_i \mathbf{Q}_i$ where $\mathbf{Q}_i \in \mathbb{R}^{D \times D}$ (ignore the multiple headed nature of attention for simplicity)
- $\mathbf{b} = 0$, in other words, no $\mathbf{b}$
- Similar computation for $\mathbf{k}$ and $\mathbf{v}$



Example # 2: The feed forward block

$$\text{output} = \text{input} \times \mathbf{W} + \mathbf{b}$$

- $FFN(\mathbf{x}) = \max(0, \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2$
- $\mathbf{W}_1 \in \mathbb{R}^{D \times 4D}$ and $\mathbf{b}_1 \in \mathbb{R}^{4D}$
- $\mathbf{W}_2 \in \mathbb{R}^{4D \times D}$ and $\mathbf{b}_2 \in \mathbb{R}^D$



Weights and Biases

A typical weight matrix

0.52	1.60	3.97	8.52	0.52	1.60	3.97	8.52	8.52	1.60
5.56	3.32	5.21	0.25	5.56	3.32	5.21	0.25	0.25	3.32
0.98	9.58	10.20	0.54	0.98	9.58	10.20	0.54	0.54	9.58
5.25	8.56	12.30	54.21	5.25	8.56	12.30	54.21	54.21	8.56
0.85	63.65	8.25	5.32	0.85	63.65	8.25	5.32	5.32	63.65
0.52	1.60	3.97	8.52	0.52	1.60	3.97	8.52	8.52	1.60
5.56	3.32	5.21	0.25	5.56	3.32	5.21	0.25	0.25	3.32
0.98	9.58	10.20	0.54	0.98	9.58	10.20	0.54	0.54	9.58
5.25	8.56	12.30	54.21	5.25	8.56	12.30	54.21	54.21	8.56
0.85	63.65	8.25	5.32	0.85	63.65	8.25	5.32	5.32	63.65

A typical bias vector

5.25	8.56	12.30	54.21	5.25	8.56	12.30	54.21	54.21	8.56
------	------	-------	-------	------	------	-------	-------	-------	------

Weights and Biases

A typical weight matrix

0.52	1.60	3.97	8.52	0.52	1.60	3.97	8.52	8.52	1.60
5.56	3.32	5.21	0.25	5.56	3.32	5.21	0.25	0.25	3.32
0.98	9.58	10.20	0.54	0.98	9.58	10.20	0.54	0.54	9.58
5.25	8.56	12.30	54.21	5.25	8.56	12.30	54.21	54.21	8.56
0.85	63.65	8.25	5.32	0.85	63.65	8.25	5.32	5.32	63.65
0.52	1.60	3.97	8.52	0.52	1.60	3.97	8.52	8.52	1.60
5.56	3.32	5.21	0.25	5.56	3.32	5.21	0.25	0.25	3.32
0.98	9.58	10.20	0.54	0.98	9.58	10.20	0.54	0.54	9.58
5.25	8.56	12.30	54.21	5.25	8.56	12.30	54.21	54.21	8.56
0.85	63.65	8.25	5.32	0.85	63.65	8.25	5.32	5.32	63.65

A typical bias vector

5.25	8.56	12.30	54.21	5.25	8.56	12.30	54.21	54.21	8.56
------	------	-------	-------	------	------	-------	-------	-------	------

A single parameter

Represented using a predetermined # of bits

Representing float parameters



- Until the models got too large, it was typical to present parameters in 32 bits
- Let us first understand how the 32-bit representation work
- Then we will study the optimizations that led to other representations

Representing float parameters



The sign bit

- Set to 1 if we are representing a negative number
- 0 for a positive number

Anatomy of a float



Exponent (E)

- 8 bits $\rightarrow 2^8 = 256$ values [0, 255]
- **0 and 255 reserved** for special numbers (e.g., NaN)

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	1
.	.	.	.	.	.	.	.
0	1	1	1	1	1	1	1
.	.	.	.	.	.	.	.
1	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1

Anatomy of a float



Exponent (E)

- 8 bits $\rightarrow 2^8 = 256$ values [0, 255]
- **0 and 255 reserved** for special numbers (e.g., NaN)
- **Usable range has 254** values [1, 254]

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	1
.	.	.	.	.	.	.	.
0	1	1	1	1	1	1	1
.	.	.	.	.	.	.	.
1	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1

Anatomy of a float



Exponent (E)

- **Biased representation:** 127 is the 0 point
- The power represented is 2^{E-127}
- $E = 1 \rightarrow 2^{-126}$ | $E = 127 \rightarrow 2^0$ | $E = 254 \rightarrow 2^{127}$
- Range is $[2^{-126}, 2^{127}] \approx [10^{-38}, 10^{38}]$

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	1
.	.	.	.	.	.	.	.
0	1	1	1	1	1	1	1
.	.	.	.	.	.	.	.
1	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1

Anatomy of a float



Fraction

- 23 bits available (24 in practice*)
- Can represent $2^{24} = 16,777,216$ values
- So a bit more than 7 digits

* In practice 24 bits due to the implicit leading bit. See the [Wikipedia page on FP-32](#)

Anatomy of a float



Putting it together

- 32-bits $[b_{31}, \dots, b_0]$ represent a number as

$$(-1)^{b_{31}} \times 2^{(b_{30} \dots b_{23} - 127)_2} \times (1.b_{22} \dots b_0)_2$$

Key facts

8 bit biased exponent $\rightarrow [10^{-38}, 10^{38}]$

24 bit fraction $\rightarrow$ 7 digit precision

Quick knowledge test

- Can we represent π in FP32 without losing information?

$$\pi = 3.14159265358979323846264338327950288419716939937510...$$

- Can we represent 1.1×10^{26} ?
- How about 1.1×10^{49} ?
- How about 1.1×10^{-20} ?
- How about 0.000000000000000000000000432?
- How about 9000000000000000000000000?

Recap: RAM requirements for LLMs

- I have a 7 billion parameter model. How much RAM do I need to host it with FP32?
- RAM Usage = 7 billion params x 4 bytes per parameter = 28 billion bytes $\approx$ **28 GB**
- I have a 405 billion parameter model. How much RAM do I need to host it with FP32?
- RAM Usage = 405 billion params x 4 bytes per parameter $\approx$ **1.6 TB**
- **Idea:** Represent numbers with lower number of bits

FP32 → FP16



Exponent (E)

- **Biased representation:** 15 is the 0 point
- The power represented is 2^{E-15}
- $E = 1 \rightarrow 2^{-14}$ | $E = 15 \rightarrow 2^0$ | $E = 30 \rightarrow 2^{15}$
- Range is $[2^{-14}, 2^{15}] \approx [10^{-4}, 10^4]$

Range is number smaller than FP32 $[10^{-38}, 10^{38}]$

Peculiarities of deep models

- Parameters can be quite small or large (often on the small side)
- For instance, 10^{-6}
- The precision of the fraction itself doesn't matter that much
- **Idea:** Use 16 bits, but use more bits for the **exponent**
- That is where **bfloat16** comes in
- Developed at Google Brain

bfloat16



Exponent (E)

- **Biased representation:** 127 is the 0 point
- The power represented is 2^{E-127}
- $E = 1 \rightarrow 2^{-126}$ | $E = 127 \rightarrow 2^0$ | $E = 254 \rightarrow 2^{127}$
- Range is $[2^{-126}, 2^{127}] \approx [10^{-38}, 10^{38}]$

Same range as FP32 $[10^{-38}, 10^{38}]$

Exercise

Representing data using int16



- Can represent $2^{16} - 1$ values
- Range from $[-32K, 32K]$
- Two's complement notation where 0 is not counted twice

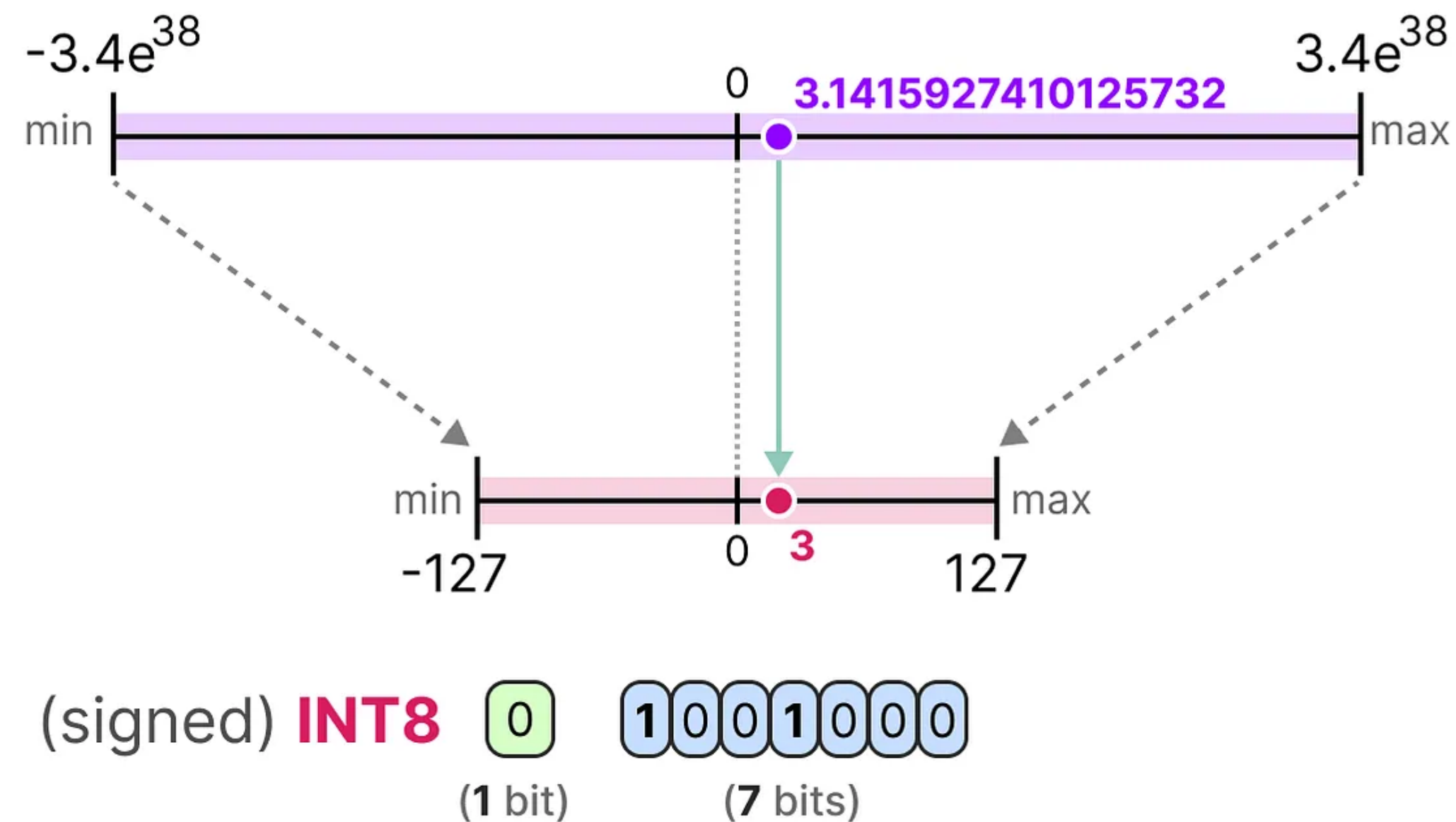
Knowledge check: What is the range of int32?

Representing data using int8



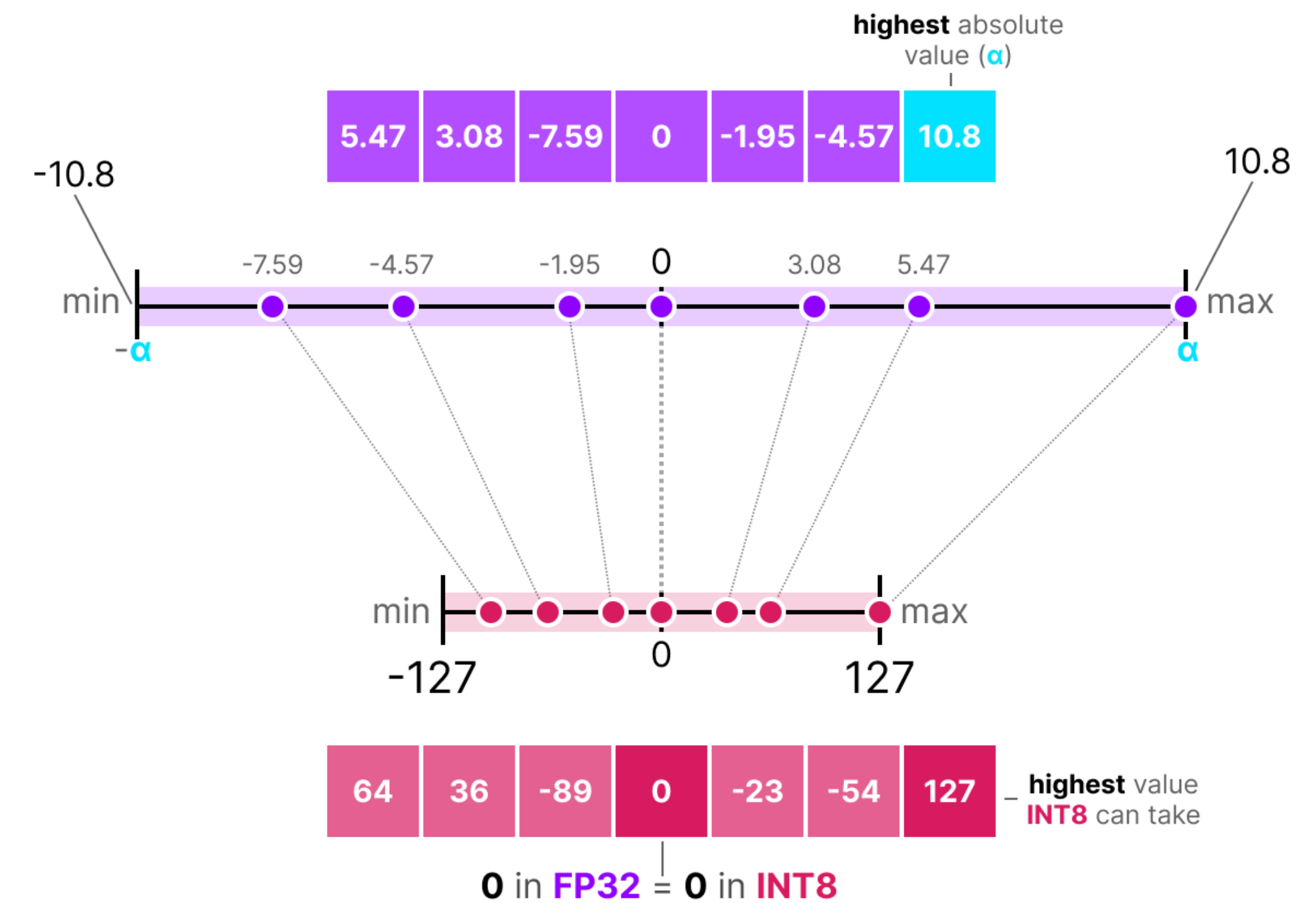
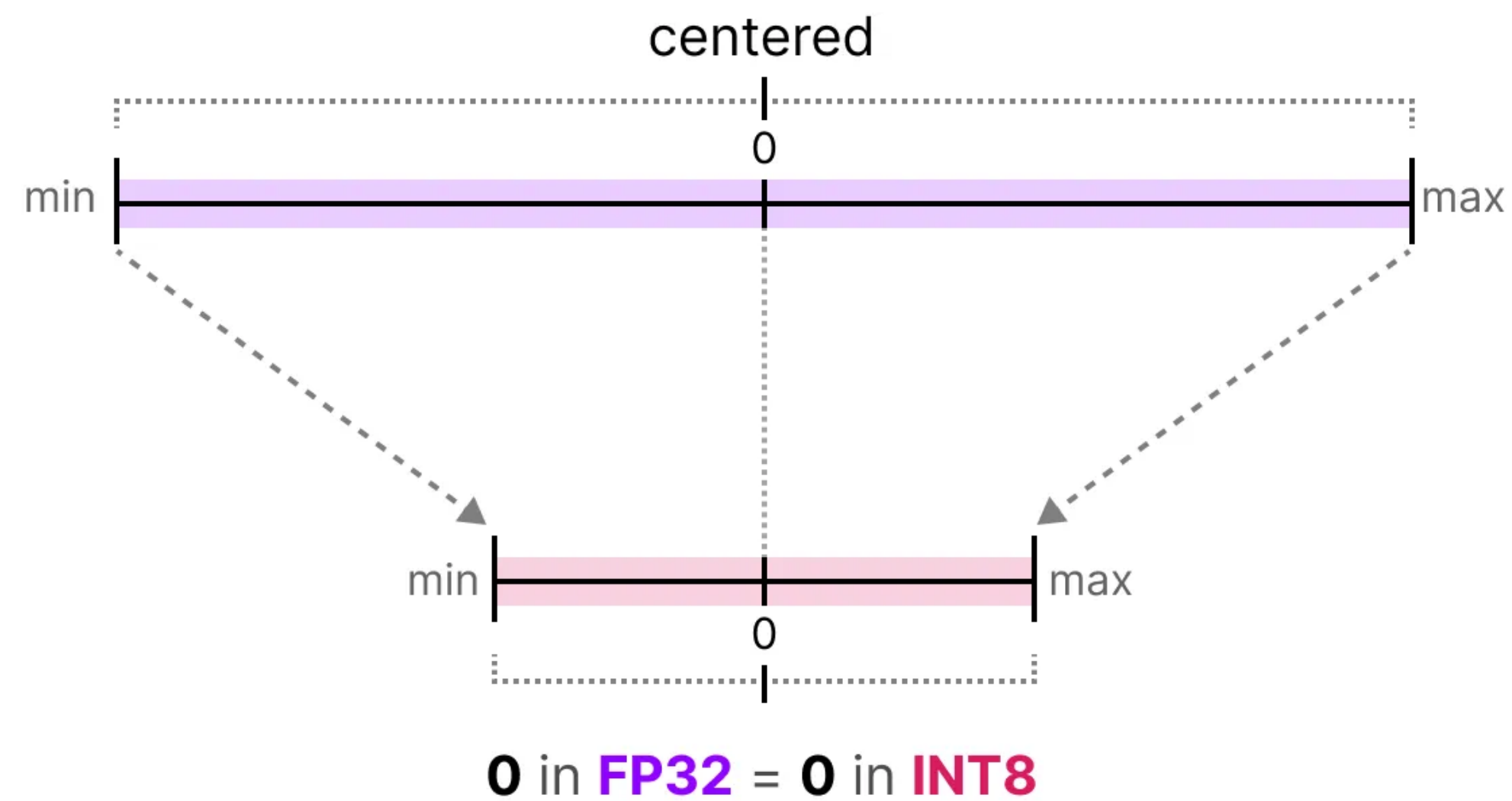
- Can represent $2^8 - 1 = 255$ values
- Range from $[-128, 127]$

Quantizing weights to int8



- Whole range between -10^{38} and 10^{38} gets mapped to 254 distinct values
- Using 8 bit ints has advantages
 - Half as much space as bfloat16
 - Can move the data quicker between RAM → GPU VRAM → computation cores
 - Hardware optimizations that can perform int8 operations much faster

Symmetric quantization



- 0 points of base and quantized weight match
- Min. and max. values are negative of each other

Performing symmetric quantization

Quantization

$$s = \frac{2^{b-1} - 1}{\alpha} \quad \text{(scale factor)}$$

$$x_{\text{quantized}} = \text{round}(s \cdot x) \quad \text{(quantization)}$$

$$s = \frac{127}{10.8} = 11.76 \quad \text{(scale factor)}$$

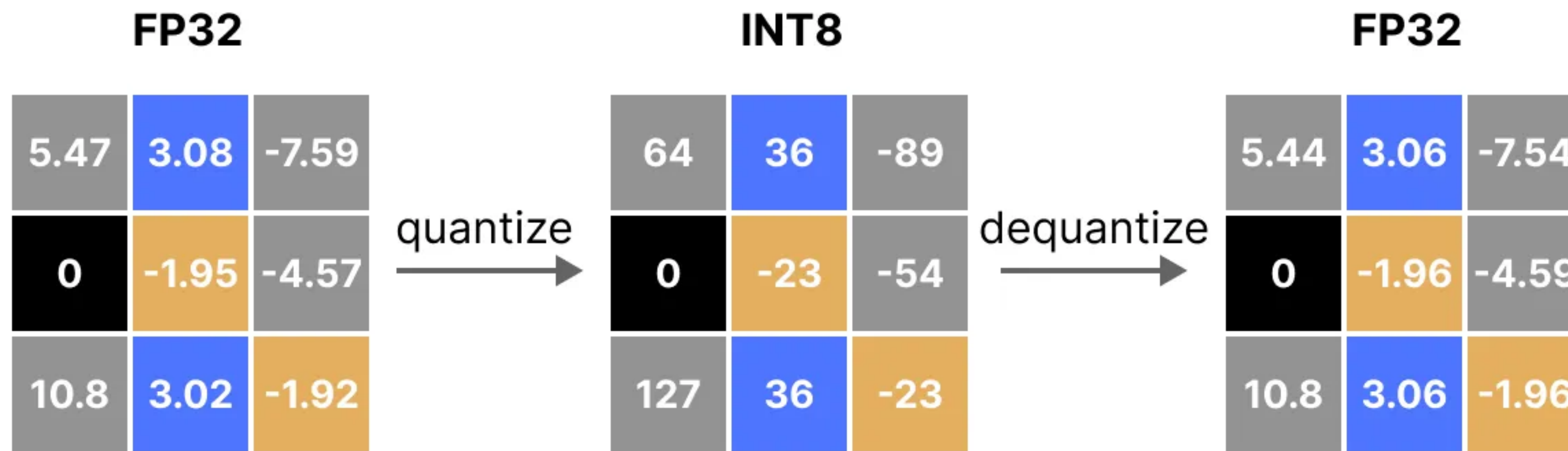
$$x_{\text{quantized}} = \text{round}(11.76 \cdot \text{■■■■■}) \quad \text{(quantization)}$$

De-quantization

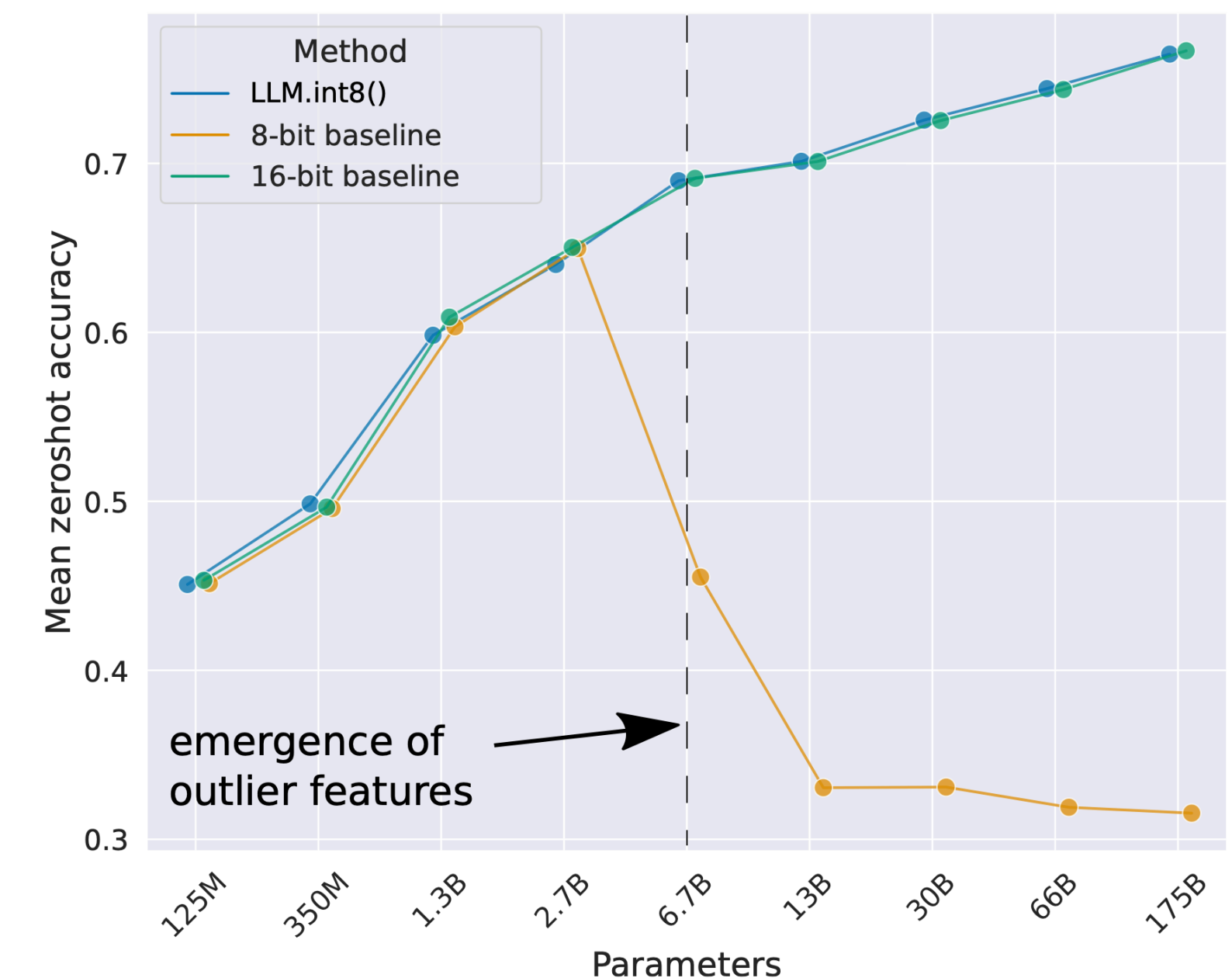
$$x_{\text{dequantized}} = \frac{\text{■■■■■}}{s} \quad \text{(dequantize)}$$

- α is the value with the largest magnitude

Quantization error



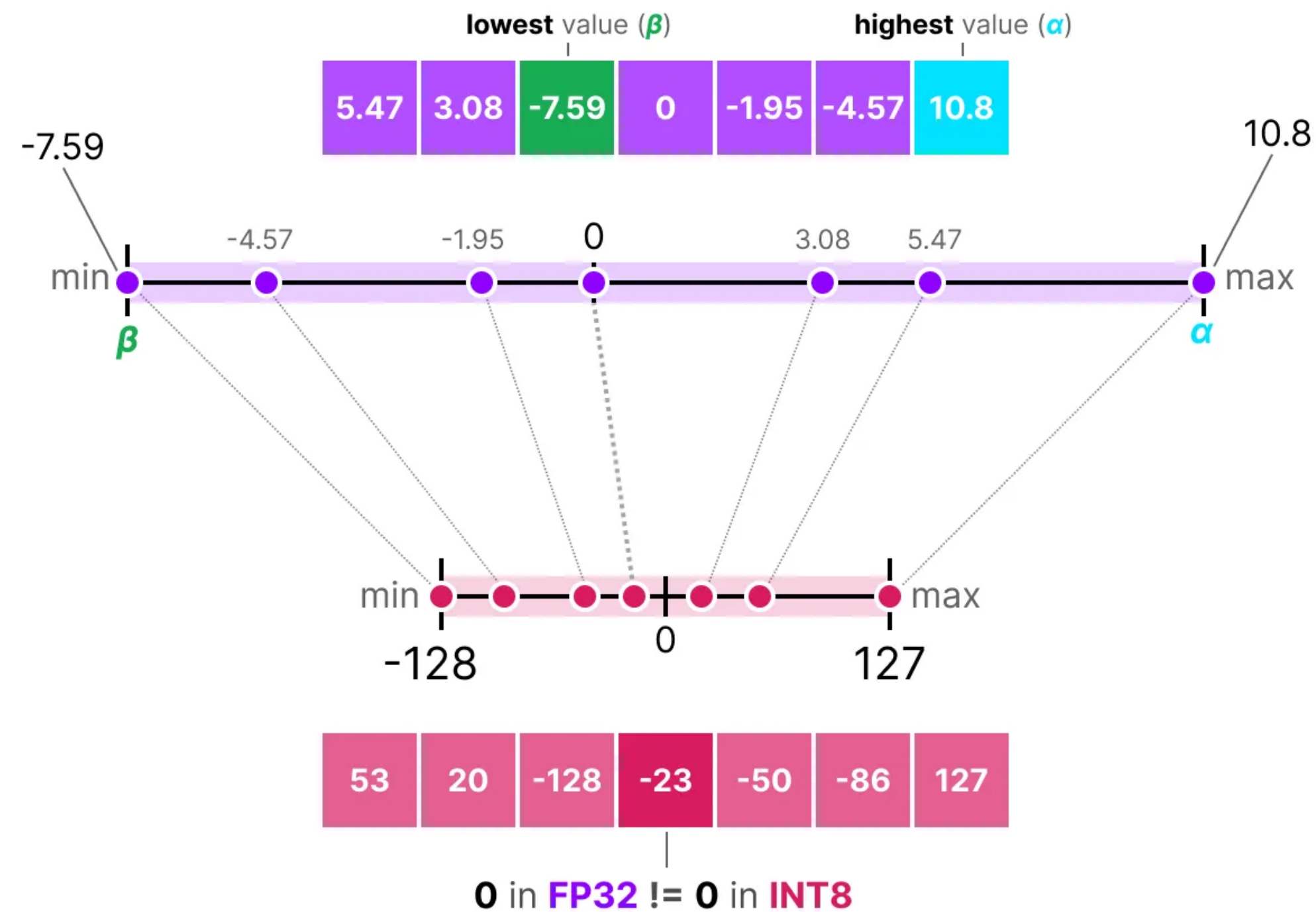
- $(3.08, 3.02) \rightarrow (36, 36) \rightarrow (3.06, 3.06)$
- Distinct values get mapped at the same quantized value
- All values have some error
- In practice, the error doesn't make a big difference in performance



[Dettmers et al]

[Grootendorst]

Asymmetric quantization



- Offers more precision than symmetric quantization
- 0s do not match

Performing asymmetric quantization

Quantization

$$s = \frac{128 - -127}{\alpha - \beta}$$

(scale factor)

$$z = \text{round}(-s \cdot \beta) - 2^{b-1}$$

(zeropoint)

$$x_{\text{quantized}} = \text{round}(s \cdot x + z)$$

(quantization)

$$s = \frac{255}{10.8 - -7.59} = 13.86$$

(scale factor)

$$z = \text{round}(-13.86 \cdot -7.59) - 128 = -23$$

(zeropoint)

$$x_{\text{quantized}} = \text{round}(13.86 \cdot \text{■■■■■} + -23)$$

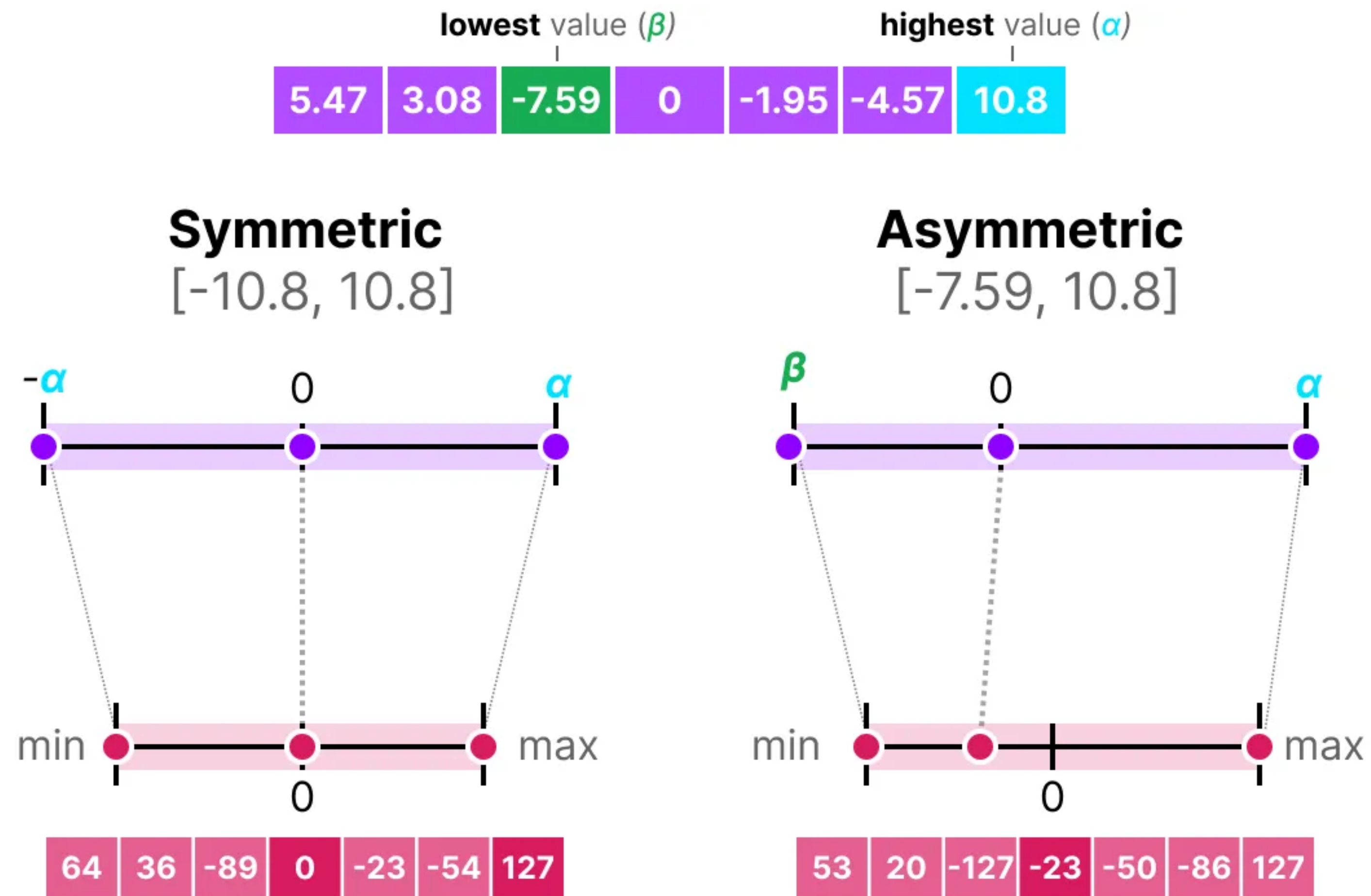
(quantization)

De-quantization

$$x_{\text{dequantized}} = \frac{\text{■■■■■} - z}{s}$$

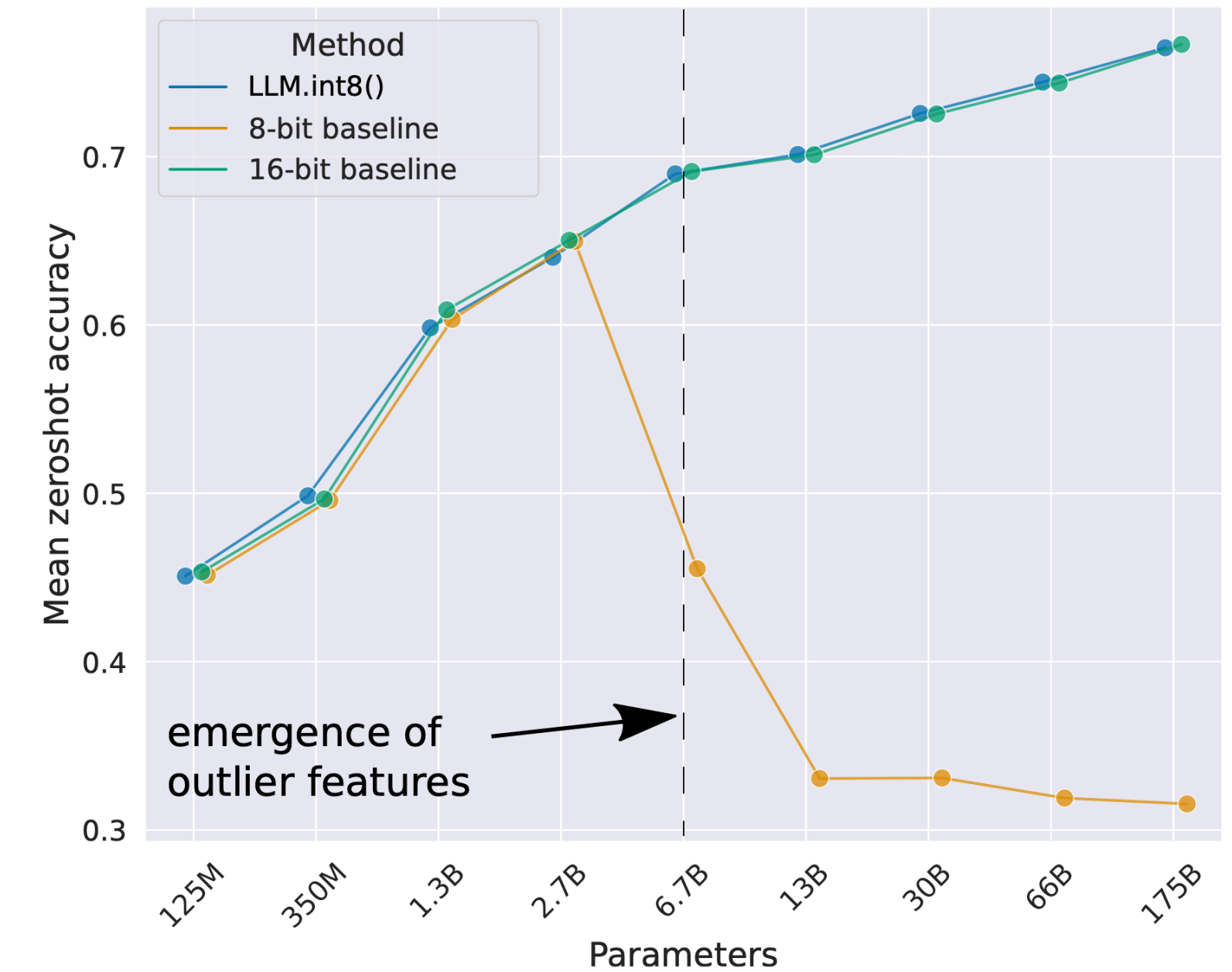
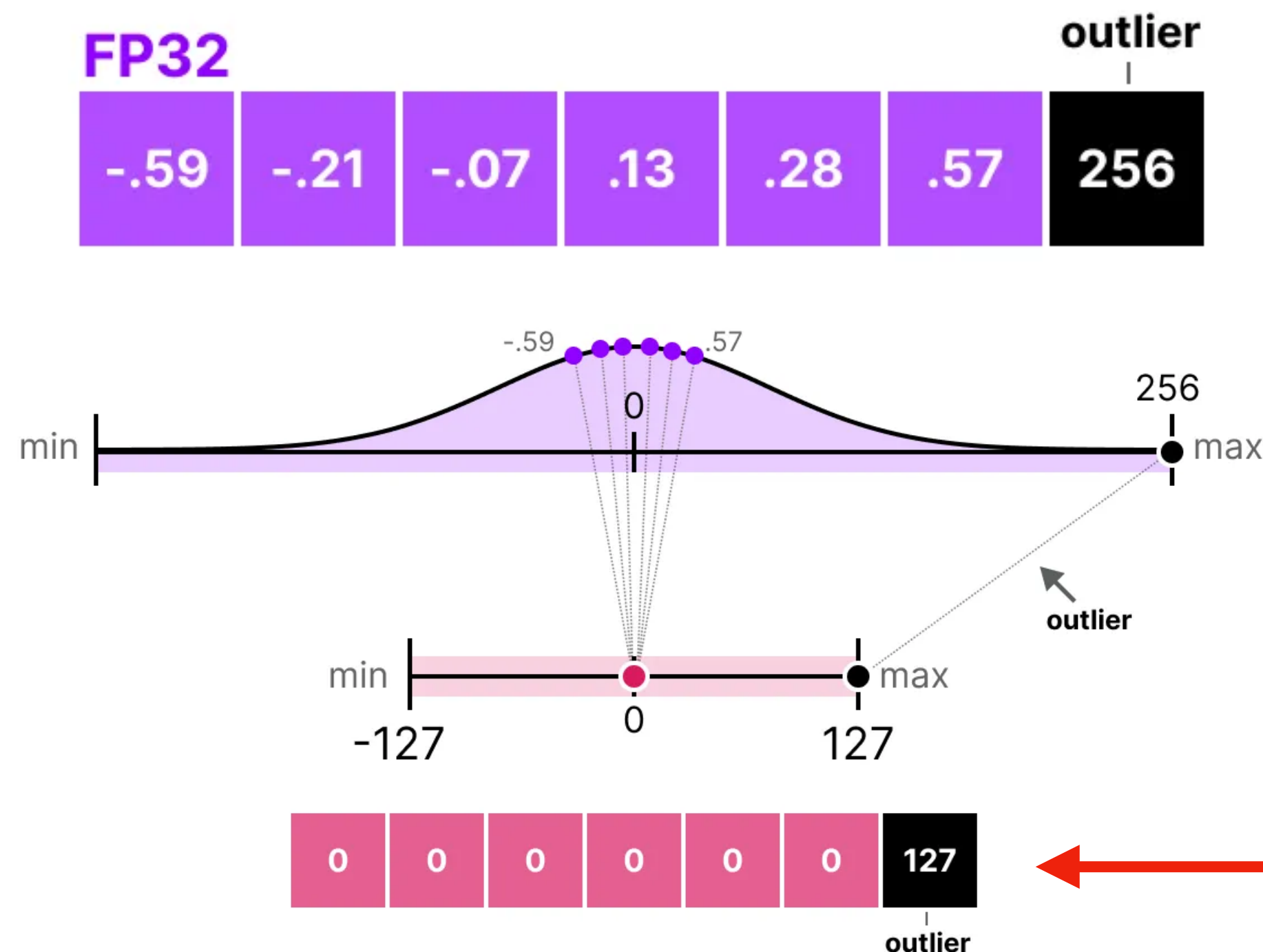
(dequantize)

Symmetric vs. asymmetric quantization



Outliers

- Outliers can take precious quantization space
- Large impact on model performance
- Start emerging at model scaled beyond 1B parameters
- Prevalence and intensity gets worse with larger sizes

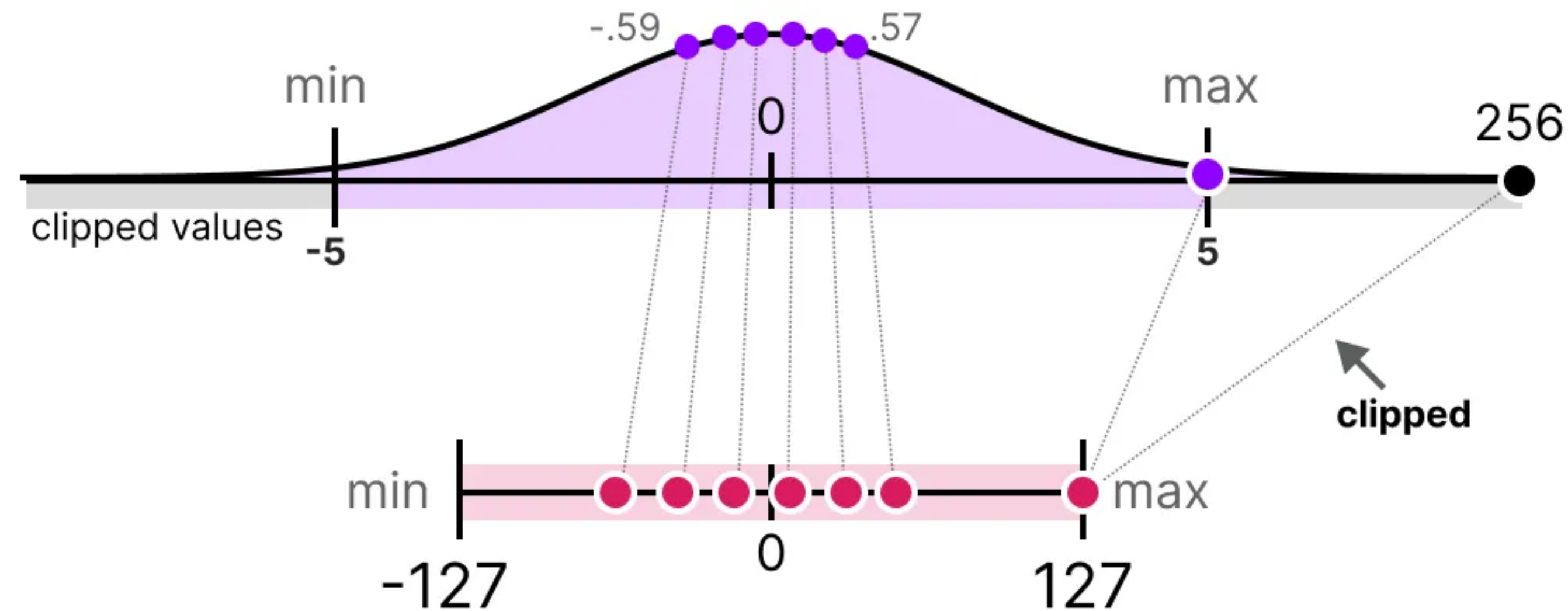


Useful weights all get zeroed out

[Dettmers et al]

[Grootendorst]

Tackling outliers with clipping



- Clip the weights to a pre-determined range, in this case $[-5, 5]$
- Any value outside the range gets mapped to the endpoints, e.g., 256 becomes 5
- Can learn using data, e.g., clip the top 5 percentiles

A lot more to quantization

- Quantization to 4 bits
- Quantizing different weights differently
 - Some weights are more important than others
- Quantizing weights but not biases
 - Weight matrix $D \times D$, bias vector $1 \times D$
- Will see more in the next lecture

Exercise

References

- [Attention is all you need](#)
- [A Visual Guide to Quantization](#)
- Wikipedia page on [Single-precision floating-point format](#)
- [What Every User Should Know About Mixed Precision Training in PyTorch](#)
- [Quantization Concepts](#)
- [LLM.int8\(\): 8-bit Matrix Multiplication for Transformers at Scale](#)